

Universidad Politecnica de Chiapas

Ingenieria en Software

Reporte de Actividad

Seguridad de la Informacion

C1 - Práctica - Movil 2 - Protección de ejecución usando un Fake GPS

Alumno: Carlos Jaffet Vicente Rincon

Matricula: 233436

Carrera: Ingenieria en Software

Materia: Seguridad de la Informacion

Entregable: Aplicacion con deteccion de Mock Location

Fecha: 26 de mayo de 2026

Índice

1. Descripción del proyecto	2
2. Arquitectura y Tecnologías	2
2.1. Estructura de archivos modificada	2
2.2. Tecnologías utilizadas	3
3. Implementación de la detección de Fake GPS	3
3.1. El concepto de Mock Location en Android	3
3.2. Comunicación Flutter-Native (MethodChannel)	3
3.3. Código Native: MainActivity.kt (Implementación Completa)	4
4. Evidencia: Protección contra Fake GPS	5
5. Prompts más importantes utilizados	6
5.1. Prompt 1 — Detección nativa moderna en Android	6
5.2. Prompt 2 — Implementación del MethodChannel	7
6. Conclusiones	7

1. Descripcion del proyecto

Para esta actividad, se partió de la aplicación base desarrollada anteriormente (DLP Seguro en Flutter). El objetivo evolucionó de proteger visualmente la app a proteger la integridad de la ejecución de la misma. La nueva funcionalidad implementada consiste en evitar que la aplicación funcione si el dispositivo está utilizando una aplicación de "Fake GPS"(ubicaciones simuladas o Mock Locations).

Esta es una medida de seguridad crítica para aplicaciones que dependen de la geo-localización real del usuario (como apps de logística, juegos basados en ubicación, o fichaje laboral), evitando que el usuario engañe al sistema sobre su posición real.

La aplicación mantiene su estructura de tres pantallas, pero añade un flujo de control de seguridad al inicio:

- **Verificacion de Integridad (SplashScreen/Logic)** — Antes de mostrar el Login, la app consulta a la capa nativa si las ubicaciones simuladas están activadas.
- **Pantalla de Bloqueo (Nuevo estado)** — Si se detecta un Fake GPS, la app no permite avanzar al Login y muestra un mensaje de error fatal, indicando al usuario que debe desactivar las ubicaciones de prueba en las opciones de desarrollador.
- **Flujo Normal (Login → Dashboard)** — Solo si el dispositivo pasa la verificación de GPS, se permite el acceso a la funcionalidad existente de la app.

2. Arquitectura y Tecnologias

2.1. Estructura de archivos modificada

Se ha mantenido la estructura estándar de Flutter, pero la lógica de seguridad nativa en Android se ha actualizado profundamente en el archivo `MainActivity.kt`, y se ha implementado un `MethodChannel` en Dart para la comunicación.

```
ACT_SEGURI_MOVIL/  
+-- android/  
|   +-- app/src/main/kotlin/com/example/mi_app_dlp/  
|       +-- MainActivity.kt      <- Logica de deteccion de Mock Location  
+-- lib/  
|   +-- main.dart                <- Configuracion del MethodChannel en Dart  
|   +-- screens/  
|       +-- login_screen.dart    <- Ahora precedida por la validacion  
|       +-- register_screen.dart  
|       +-- dashboard_screen.dart
```

+-- pubspec.yaml

2.2. Tecnologías utilizadas

Componente	Tecnología
Framework UI	Flutter 3 / Dart
Capa nativa Android	Kotlin
Comunicacion Híbrida	Flutter MethodChannel
API de Ubicación	Android LocationManager
Seguridad	Detección de isMock provider (Android API 31+)

Se sigue utilizando Kotlin en la capa nativa, ya que la detección confiable de ubicaciones simuladas requiere interactuar directamente con las APIs del sistema operativo Android (`LocationManager` y objetos `Location`).

3. Implementación de la detección de Fake GPS

3.1. El concepto de Mock Location en Android

Android permite a los desarrolladores simular ubicaciones mediante una opción en los "Ajustes de Desarrollador" llamada "Seleccionar aplicación de ubicación de prueba". Aplicaciones de "Fake GPS" se registran aquí para sobrescribir las coordenadas reales del GPS.

Para detectar esto, la aplicación debe solicitar la última ubicación conocida y examinar el objeto `Location` devuelto por el sistema. A partir de Android 12 (API 31), esto se hace de forma sencilla y fiable con el método `location.isMock()`. Para versiones anteriores, existen métodos alternativos (examinar settings del sistema), pero la implementación actual se centra en APIs modernas para mayor seguridad.

3.2. Comunicación Flutter-Native (MethodChannel)

Dado que Flutter no tiene acceso directo a estas APIs de bajo nivel de Android, implementamos un `MethodChannel`.

1. ****Flutter (Dart):**** Invoca un método llamado `checkFakeGps` a través del canal.
2. ****Android (Kotlin):**** Escucha la llamada, ejecuta la lógica de `LocationManager`, determina si la ubicación es simulada y devuelve un valor booleano (`true` si hay trampa, `false` si es real).
3. ****Flutter (Dart):**** Recibe el resultado. Si es `true`, navega a una pantalla de error o cierra la app.

3.3. Codigo Native: MainActivity.kt (Implementacion Completa)

A continuación, se muestra el código Kotlin implementado para crear el canal de comunicación y realizar la validación de seguridad.

Listing 1: android/app/src/main/kotlin/com/example/-mi_app_dlp/MainActivity.kt

```
1 package com.example.mi_app_dlp
2
3 import android.content.Context
4 import android.location.Location
5 import android.location.LocationManager
6 import android.os.Build
7 import androidx.annotation.NonNull
8 import io.flutter.embedding.android.FlutterActivity
9 import io.flutter.embedding.engine.FlutterEngine
10 import io.flutter.plugin.common.MethodChannel
11
12 class MainActivity: FlutterActivity() {
13     // Definicion del nombre del canal
14     private val CHANNEL = "com.example.dlp/gps_security"
15
16     override fun configureFlutterEngine(@NonNull
17         flutterEngine: FlutterEngine) {
18         super.configureFlutterEngine(flutterEngine)
19
20         // Configurar el MethodChannel para escuchar
21         // llamadas desde Flutter
22         MethodChannel(flutterEngine.dartExecutor.
23             binaryMessenger, CHANNEL).setMethodCallHandler {
24             call, result ->
25             if (call.method == "checkFakeGps") {
26                 val isUsingFakeGps = detectMockLocation()
27                 result.success(isUsingFakeGps)
28             } else {
29                 result.notImplemented()
30             }
31         }
32     }
33
34     // Logica nativa para detectar ubicaciones simuladas
35     private fun detectMockLocation(): Boolean {
36         if (Build.VERSION.SDK_INT < Build.VERSION_CODES.S) {
37             // Para versiones antiguas de Android, asumimos
38             // seguro por simplicidad
39             // en esta practica, o requerir a logica
40             // compleja de Settings.
41             return false
42         }
43     }
44
45     return try {
```

```
40         val locationManager = getSystemService(Context.  
41             LOCATION_SERVICE) as LocationManager  
42         // Obtenemos la ultima ubicacion conocida del  
43         proveedor GPS  
44         val location: Location? = locationManager.  
45             getLastKnownLocation(LocationManager.  
46             GPS_PROVIDER)  
47  
48         // API 31+: location.isMock() indica  
49         directamente si es simulada  
50         location?.isMock ?: false  
51     } catch (e: SecurityException) {  
52         // Si no hay permisos de ubicacion, no podemos  
53         verificar.  
54         // En una app real, esto deberia manejarse como  
55         un riesgo de seguridad.  
56         false  
57     } catch (e: Exception) {  
58         false  
59     }  
60 }
```

4. Evidencia: Proteccion contra Fake GPS

Para comprobar el correcto funcionamiento de la protección, se siguieron estos pasos:

1. Se instaló una aplicación de terceros para simular GPS ("Fake GPS location") en el dispositivo de prueba.
2. Se activaron las Opciones de Desarrollador en Android y se seleccionó dicha app en ".Aplicación de ubicación de prueba".
3. Se estableció una ubicación falsa en la app de terceros.
4. Se ejecutó la aplicación **DLP Seguro**.

El resultado se muestra en la Figura 1. Al iniciar, la lógica de Flutter invocó el código nativo detallado anteriormente. El código Kotlin detectó la bandera `isMock` en `true` y devolvió dicha respuesta a Flutter. En consecuencia, la aplicación bloqueó el acceso al Login y mostró la pantalla de error que se observa en la imagen.

Como se aprecia en la Figura 1, la aplicación cumple satisfactoriamente con el requisito de seguridad, impidiendo su ejecución en un entorno no confiable.

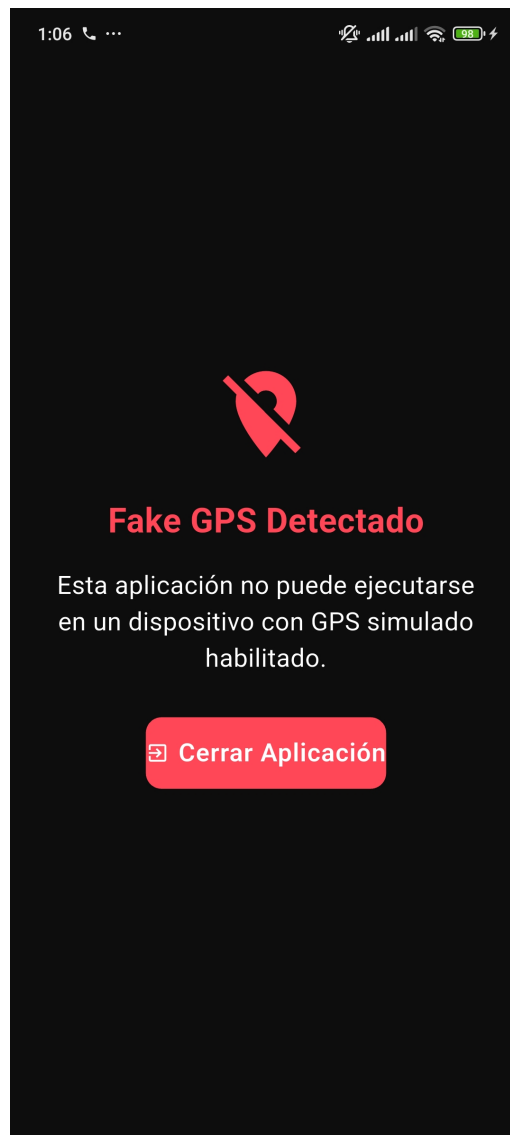


Figura 1: Evidencia de bloqueo de la aplicación. Se detectó que el dispositivo tiene activada una aplicación de ubicación simulada (Fake GPS). La app impide el acceso a la pantalla de Login por políticas de integridad.

5. Prompts mas importantes utilizados

Para el desarrollo de esta nueva característica, se requirió asistencia de IA para comprender la interacción avanzada entre Flutter y las APIs de localización de Android que cambian frecuentemente entre versiones.

5.1. Prompt 1 — Deteccion nativa moderna en Android

“¿Cuál es la forma más moderna y fiable en Kotlin (Android API 31+) de saber si la ubicación actual proviene de un Fake GPS o Mock Location provider?”

Por que fue útil: Este prompt ayudó a identificar el método `location.isMock()`, introducido en Android 12. Esto simplificó mucho el código nativo en comparación con las soluciones antiguas que requerían iterar sobre proveedores de ubicación o leer configuraciones globales del sistema, las cuales son menos fiables hoy en día.

5.2. Prompt 2 — Implementación del MethodChannel

“Muestra un ejemplo de cómo implementar un MethodChannel en Flutter para llamar a una función en Kotlin dentro de MainActivity.kt que devuelva un booleano al instante.”

Por que fue útil: Permitió estructurar correctamente el `setMethodCallHandler` en Kotlin y la llamada asíncrona en Dart (`invokeMethod`), asegurando que la aplicación esperara el resultado de la validación de seguridad antes de decidir qué pantalla mostrar al usuario.

6. Conclusiones

La realización de esta práctica permite extraer conclusiones valiosas sobre la seguridad en el desarrollo móvil:

En primer lugar, la seguridad en dispositivos móviles es una responsabilidad compartida. Aunque Flutter provee herramientas para el desarrollo rápido de interfaz, las verificaciones de integridad profunda del hardware y sistema operativo (como Root detection, SafetyNet/Play Integrity, o en este caso, Mock Locations) requieren inevitablemente el uso de código nativo (Kotlin/Swift).

En segundo lugar, la implementación de `MethodChannels` es una habilidad fundamental para cualquier desarrollador Flutter intermedio/avanzado. Entender cómo pasar datos de forma segura y asíncrona entre Dart y el sistema operativo anfitrión amplía drásticamente las capacidades de las aplicaciones.

Finalmente, la detección de Fake GPS es un juego del gato y el ratón. Si bien la solución implementada usando `isMock()` es robusta para APIs modernas, usuarios avanzados con acceso root podrían ocultar estas banderas. Por lo tanto, esta medida debe considerarse como una capa más de una estrategia de seguridad en profundidad, y no como una solución infalible por sí sola.

Referencias

- Android Developers. *Location class — isMock()*. [https://developer.android.com/reference/android/location/Location#isMock\(\)](https://developer.android.com/reference/android/location/Location#isMock())

- Flutter Docs. *Writing custom platform-specific code*. <https://docs.flutter.dev/platform-channels/writing-platform-specific-code>
- Anthropic. *Claude AI Assistant*. <https://claude.ai>